

OLLAMA-NAN-WEDGE

The ollama embedding NaN wedge: root cause and fix

Status: root-caused, reproduced on demand, fix identified and validated. **Date:** 2026-08-26 / 27 **Host:** nezha, Intel Iris Xe Graphics (TGL GT2), i915 + Mesa Vulkan, 62.6 GiB RAM **Backend:** ollama 0.23.4 (ollama-0.23.4-alt1.x86_64), model ordiis/jina-embeddings-v2-base-code

1. Verdict up front

Vulkan/GPU is the cause. CPU-only completely avoids it.

The wedge is not a request-count or a “sustained load” phenomenon. It is triggered by a **single embedding request whose token count fills the 8192-token context**. Such a request produces a Vulkan compute dispatch that the i915 kernel driver cannot complete inside its fence timeout. The kernel logs `Fence expiration time out`
`i915-...:ollama[pid]`, the dispatch is abandoned, and the model returns garbage. Repeat it a few times and the Vulkan context degrades permanently: every subsequent embed — including “hi” — returns `HTTP 500 {"error":"failed to encode response: json: unsupported value: NaN"}` until the runner process is destroyed (`ollama stop`).

The same model, same input, same ollama, with `num_gpu: 0` returns correct, bit-deterministic 768-dim vectors and produces **zero** i915 events.

There is also a **worse, silent failure mode** that nobody was looking for: before the hard NaN 500 appears, the GPU path returns **HTTP 200 with an all-zero 768-dim embedding**. Any indexer that only checks the HTTP status will happily write those into the vector store. See §4.3 — this is the finding with the largest blast radius.

Single recommended fix: stop using Vulkan for this workload. Immediately, per-request: options: {"num_gpu": 0}. Durably, host-wide: GGML_VK_VISIBLE_DEVICES=-1 in /etc/sysconfig/ollama (§7).

2. Method

2.1 Harness

nanprobe.py — no dependencies, one CSV row per request, and it snapshots the i915 kernel fence/hang counters before and after each condition so GPU faults can be correlated with the first bad response. Crucially it classifies **three** outcomes, not two: ok, NAN (HTTP 500 or non-finite values), and zerovec (HTTP 200 with an all-zero vector).

```
#!/usr/bin/env python3
"""
nanprobe.py - ollama /api/embed load harness.

Sends embed requests in a loop until the first NaN response, a
hard request cap,
or a wall-clock deadline. Logs one CSV line per request:
    idx,ts,http_status,verdict,latency_s,n_vectors,dim,chars_sent
verdict is one of: ok | NAN | zerovec | err | timeout
Also snapshots the i915 kernel fence/hang counter before and
after, so a GPU
fault can be correlated with the first NaN.

Usage:
    nanprobe.py --label NAME [--num-gpu N] [--chars N] [--batch N]
                [--max-req N] [--max-sec N] [--out FILE]
"""
import argparse, json, subprocess, sys, time, urllib.request,
    urllib.error

URL = "http://localhost:11434/api/embed"
MODEL = "ordis/jina-embeddings-v2-base-code"

# Deterministic pseudo-code text so every condition sends
    comparable content.
UNIT = ("def transform_%d(rows, key):\n"
        "    out = []\n"
```

```

        "    for r in rows:\n"
        "        if r.get(key) is not None:\n"
        "            out.append((r[key], len(r)))\n"
        "    return sorted(out)\n")

def make_text(nchars, salt):
    if nchars <= 12:
        return ("x%d" % salt)[:max(1, nchars)].ljust(min(nchars,
10), "y")
    buf, n, i = [], 0, 0
    while n < nchars:
        s = UNIT % (salt * 1000 + i)
        buf.append(s); n += len(s); i += 1
    return "".join(buf)[:nchars]

def i915_counts():
    """Count i915 fence-timeout and GPU-hang kernel lines seen so
    far."""
    try:
        out = subprocess.run(
            ["journalctl", "-k", "--no-pager", "--since", "24
            hours ago", "-o", "short-iso"],
            capture_output=True, text=True, timeout=30).stdout
    except Exception:
        return (-1, -1, "")
    fence = [l for l in out.splitlines() if
        "Fence expiration time out" in l]
    hang = [l for l in out.splitlines()
        if "GPU HANG" in l or "context reset due to GPU hang"
        in l]
    last = (fence + hang)[-1] if (fence or hang) else ""
    return (len(fence), len(hang), last)

def post(payload, timeout):
    body = json.dumps(payload).encode()
    req = urllib.request.Request(URL, data=body,
                                headers={"Content-Type":
        "application/json"})
    t0 = time.monotonic()
    try:
        with urllib.request.urlopen(req, timeout=timeout) as r:
            return r.status, r.read().decode("utf-8", "replace"),
                time.monotonic() - t0
    except urllib.error.HTTPError as e:
        return e.code, e.read().decode("utf-8", "replace"),
            time.monotonic() - t0
    except Exception as e:
        return 0, "CLIENTERR %s: %s" % (type(e).__name__, e),
            time.monotonic() - t0

```

```

def classify(status, raw):
    if "unsupported value: NaN" in raw or '"NaN"' in raw or ":
        NaN" in raw:
            return "NaN", 0, 0
    if status == 200 and '"embeddings"' in raw:
        try:
            d = json.loads(raw)
            embs = d.get("embeddings") or []
            dim = len(embs[0]) if embs and isinstance(embs[0],
list) else 0
            if embs and isinstance(embs[0], list):
                if not all(isinstance(v, (int, float)) for v in
embs[0][:8]):
                    return "NaN", len(embs), dim
                if all(v == 0 for v in embs[0]):
                    return "zerovec", len(embs), dim    # silent
corruption
                return "ok", len(embs), dim
            except Exception:
                return "NaN", 0, 0
    if status == 0:
        return "timeout" if "timed out" in raw or "TimeoutError"
in raw else "err", 0, 0
    return "err", 0, 0

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--label", required=True)
    ap.add_argument("--num-gpu", type=int, default=None,
        help="value for options.num_gpu; omit for
ollama default")
    ap.add_argument("--chars", type=int, default=400)
    ap.add_argument("--batch", type=int, default=1, help="len of
the input array")
    ap.add_argument("--max-req", type=int, default=300)
    ap.add_argument("--max-sec", type=float, default=900)
    ap.add_argument("--timeout", type=float, default=180)
    ap.add_argument("--out", default=None)
    a = ap.parse_args()

    out = open(a.out, "w", buffering=1) if a.out else sys.stdout
    f0, h0, _ = i915_counts()
    print("# label=%s num_gpu=%s chars=%d batch=%d max_req=%d
max_sec=%.0f "
        "start=%s i915_fence_before=%d i915_hang_before=%d"
        % (a.label, a.num_gpu, a.chars, a.batch, a.max_req,
a.max_sec,
        time.strftime("%Y-%m-%dT%H:%M:%S"), f0, h0),
        file=out)
    print("idx,ts,status,verdict,latency_s,n_vec,dim,chars_sent",
        file=out)

```

```

t_start, first_nan, lat, i = time.time(), None, [], 0
while i < a.max_req and (time.time() - t_start) < a.max_sec:
    i += 1
    texts = [make_text(a.chars, i * 97 + k) for k in
              range(a.batch)]
    payload = {"model": MODEL, "input": texts if a.batch > 1
              else texts[0]}
    if a.num_gpu is not None:
        payload["options"] = {"num_gpu": a.num_gpu}
    status, raw, dt = post(payload, a.timeout)
    verdict, nvec, dim = classify(status, raw)
    lat.append(dt)
    print("%d,%s,%d,%s,%.3f,%d,%d,%d"
          % (i, time.strftime("%H:%M:%S"), status, verdict,
            dt, nvec, dim,
              sum(len(t) for t in texts)), file=out)
    if verdict == "NAN" and first_nan is None:
        first_nan = i
        # Confirm the wedge is total: a trivial input must
        also fail.
        p2 = {"model": MODEL, "input": "hi"}
        if a.num_gpu is not None:
            p2["options"] = {"num_gpu": a.num_gpu}
        s2, r2, d2 = post(p2, 60)
        v2, _, _ = classify(s2, r2)
        print("# FIRST_NAN at request %d; trivial-input
        recheck -> %s (http %d, %.2fs)"
              % (i, v2, s2, d2), file=out)
        break

f1, h1, lastline = i915_counts()
n = len(lat); q = sorted(lat)
print("# SUMMARY " + json.dumps({
    "label": a.label, "num_gpu": a.num_gpu, "chars": a.chars,
    "batch": a.batch,
    "requests_sent": n, "first_nan_at": first_nan,
    "elapsed_s": round(time.time() - t_start, 1),
    "lat_first5_mean": round(sum(lat[:5]) / max(1,
        len(lat[:5])), 2) if lat else None,
    "lat_last5_mean": round(sum(lat[-5:]) / max(1,
        len(lat[-5:])), 2) if lat else None,
    "lat_median": round(q[n // 2], 2) if n else None,
    "lat_max": round(max(lat), 2) if n else None,
    "i915_fence_timeouts_during": f1 - f0,
    "i915_gpu_hangs_during": h1 - h0,
    "i915_last_event": lastline[:120],
}), file=out)

if __name__ == "__main__":
    main()

```

Driver, so every condition starts from a genuinely fresh runner:

```
#!/usr/bin/env bash
# runcond.sh LABEL [extra nanprobe args...]
set -u
S="$(cd "$(dirname "$0")" && pwd)"
MODEL=ordis/jina-embeddings-v2-base-code
LABEL="$1"; shift
ollama stop "$MODEL" >/dev/null 2>&1
sleep 5
python3 "$S/nanprobe.py" --label "$LABEL" --out "$S/cond_${LABEL}.csv" "$@"
ollama ps
```

2.2 Controlling the confound

A `lumen index` job was hitting the same backend throughout. With it running, every one of my requests queued behind its batches and took **~40 s instead of ~0.3 s** — a 130x distortion that makes any latency or throughput measurement meaningless. For the measured conditions the `lumen index` worker was therefore **SIGSTOPped** (never killed, never `systemctl restart`d), with a detached watchdog guaranteeing `SIGCONT` after 40 minutes even if the session died. It was resumed at the end. Runs contaminated by contention are marked below and excluded from conclusions.

Backend placement was never trusted from `ollama ps` alone; every condition was confirmed against the journal line `load_tensors:` offloaded N/13 layers to GPU.

3. Results

3.1 Controlled conditions

All rows: fresh runner (`ollama stop` first), uncontended, placement journal-verified.

#	Condition	Backend	Payload	Reqs sent	First failure	Latency	i915 fence timeouts
c-short	C1_gpu_short10	GPU 13/13	10 ch × 1	300	none	0.06 s median	0
a	A1_gpu_400char			299	none	0.41 s median,	0

#	Condition	Backend	Payload	Reqs sent	First failure	Latency	i915 fence timeouts
		GPU 13/13	400 ch × 1			0.42 s last-5	
d	D1_gpu_batch32	GPU 13/13	400 ch × 32 (12.8 kB/req)	50	none	9.5 s → 10.9 s	0
e	E1_gpu_large	GPU 13/13	8000 ch × 4 (32 kB/req)	7	req 1 (zerovec) → req 8 hard NaN 500	80.5 s flat	168 (24/req)
e-repeat	E2_gpu_large	GPU 13/13	8000 ch × 4	8	req 1 (zerovec) → req 8 hard NaN 500	84.2 s → 86.1 s	199 + 3 GPU HANGs
e-det	GPU determinism	GPU 13/13	8000 ch × 1, ×4	4	request 1 (zerovec, all 4)	20.1–21.5 s	yes
b	B2_cpu_large	CPU 0/13	8000 ch × 4 (32 kB/req)	3	none , all valid 768-dim	317–371 s (mean 346 s)	0 , and 0 GPU hangs
b-det	CPU determinism	CPU 0/13	8000 ch × 1, ×4	4	none , pairwise cosine 1.000000	71–129 s	0
b-calib	calib_cpu	CPU 0/13	8000 ch × 1	4	none	68–72 s	0

zerovec = HTTP 200 carrying an all-zero 768-dim vector.

Condition (a) never failed — 299 GPU requests, and 300 more at 10 chars, zero fence timeouts. **Condition (d) never failed** — 50 batch-of-32 requests, 1600 texts, 640 kB, is *not* enough to wedge it on its own.

Condition (b) is the money row. B2_cpu_large sent the *byte-for-byte identical payload* that corrupts the GPU on request 1 and wedges it by request 8. On CPU: three requests, three valid 768-dim vectors, **zero**

i915 fence timeouts, **zero** GPU hangs. It is ~4x slower per request (346 s vs 80 s) — but the GPU’s 80 s was never real work, it was four 20-second fence timeouts in a row.

The failing condition was reproduced from a fresh runner **three independent times** (E1, E2, determinism run). Silent corruption began at **request 1** in all three. In the two full-length runs the hard NaN-500 wedge landed at **request 8 in both** — the failure point is not random, it is a stable count of context-filling dispatches.

3.2 Input-size ladder — the actual trigger

Single requests, fresh GPU runner, 3 reps per size:

chars/ request	latency	i915 fence timeouts per request	result
10	0.07–0.11 s	0	ok
500	0.36–0.56 s	0	ok
2 000	2.5–5.0 s	0	ok
4 000	17.1–18.8 s	0	ok
8 000	20.1–20.3 s	6	corrupt
16 000	20.5–20.7 s	7	corrupt
32 000	20.6–21.0 s	7	corrupt

Kernel events, bucketed by timestamp, over that ladder:

```
0 (all sizes ≤ 4000 chars)
6 2026-08-26T23:40:45  <- first 8000-char request
6 2026-08-26T23:41:05
6 2026-08-26T23:41:26
7 2026-08-26T23:41:47  <- 16000-char
7 2026-08-26T23:42:07
...
```

Two things fall out of this table and they are the whole story:

1. **Fence timeouts begin exactly at the 8000-char step** and then fire on *every* subsequent large request. Below that: zero, across 599 requests.
2. **Latency saturates at ~20.5 s and stops responding to input size.** 8 000, 16 000 and 32 000 chars all take the same ~20.5 s. That is not compute — the input is truncated to the 8192-token context, and the plateau *is the i915 fence timeout expiring*. The GPU work never

finishes; the driver gives up and the pipeline moves on with whatever was in the buffer.

ordis/jina-embeddings-v2-base-code has an 8192-token context and the runner is loaded with BatchSize:8192 KvSize:8192. ~8000 characters of source code is where the batch fills the context, so a single dispatch becomes large enough to blow the fence timeout.

3.3 Escalation to the permanent wedge

E1_gpu_large, verbatim:

```
# label=E1_gpu_large num_gpu=None chars=8000 batch=4 ...
i915_fence_before=175
idx,ts,status,verdict,latency_s,n_vec,dim,chars_sent
1,23:45:28,200,zerovec,82.046,4,768,32000
2,23:46:48,200,zerovec,80.585,4,768,32000
3,23:48:09,200,zerovec,80.586,4,768,32000
4,23:49:30,200,zerovec,80.499,4,768,32000
5,23:50:50,200,zerovec,80.464,4,768,32000
6,23:52:10,200,zerovec,80.510,4,768,32000
7,23:53:31,200,zerovec,80.484,4,768,32000
# SUMMARY {... "i915_fence_timeouts_during": 168 ...}
```

Immediately afterwards, on the same runner:

```
$ curl -s localhost:11434/api/embed -d
'{"model":"...", "input":"hi"}'
{"error":"failed to encode response: json: unsupported value:
NaN"}
http=500 t=0.0596
```

0.06 s — it is not even attempting to compute. The runner is dead and stays dead until `ollama stop`. **Every request returned HTTP 200 right up until the runner was already destroyed.** The 500 is the *tombstone*, not the fault.

The repeat run E2_gpu_large landed on the same number independently — $7 \times$ zerovec, then:

```
8,00:45:xx,500,NAN,...
# FIRST_NAN at request 8; trivial-input recheck -> NAN (http 500,
16.54s)
# SUMMARY {... "first_nan_at": 8, "i915_fence_timeouts_during":
199,
#           "i915_gpu_hangs_during": 3,
#           "i915_last_event": "...[drm] ollama[2551406] context
reset due to GPU hang"}
```

199 fence timeouts and 3 kernel-level GPU hangs in 11 minutes, ending in context reset due to GPU hang — the kernel confirming the device was reset out from under the runner. Two independent runs, same trigger, same first-NaN request number: 8.

3.4 Historical natural experiment (12 h of journal, before I touched anything)

Six runner loads, reconstructed from runner.go:895 load lines, /api/embed GIN lines and llm embedding error lines:

Runner loaded	GPULayers	HTTP 200 before 1st NaN	First NaN	NaN log lines
16:10:38	13 (GPU)	9	16:14:38	93
20:38:53	13 (GPU)	144	never (evicted first)	0
22:34:22	13 (GPU)	28	23:01:35	380
23:06:05	13 (GPU)	4	23:08:24	96
23:11:19	13 (GPU)	3	never (replaced 2 min later)	0
23:13:06	0 (CPU)	19+	never	0

All **657** NaN events in 12 hours occurred inside GPU-offloaded runners. The one CPU-only runner logged zero. The wildly varying “requests before failure” (9, 28, 4, and 144-without- failure) is exactly what §3.2 predicts: it depends on *when a context-filling chunk happens to come up in the corpus*, not on how many requests have gone by.

The smoking gun is the timing:

```
Aug 26 16:14:34 kernel: Fence expiration time out
i915-0000:00:02.0:ollama[3538599]:2a60!
Aug 26 16:14:38 ollama: llm embedding error: ... json: unsupported
value: NaN
```

Four seconds. And later, the full escalation:

```
Aug 26 22:34:14 i915 [drm] Resetting rcs0 for preemption time out
Aug 26 22:34:14 i915 [drm] ollama[4101609] context reset due to
GPU hang
Aug 26 22:34:15 i915 [drm] GPU HANG: ecode 12:1:859ffffb, in
```

4. What is actually happening

4.1 Mechanism

1. A chunk arrives that fills the 8192-token context (~8 kB of source).
2. ggml-vulkan issues one large dispatch to the Intel Iris Xe via Mesa/anv.
3. The dispatch exceeds the i915 fence wait timeout (~20 s). Kernel: Fence expiration time out. The result buffer is never validly written.
4. ollama reads that buffer anyway. It contains zeros or non-finite values.
5. **Zeros** → ollama's normalisation leaves them as zeros → **HTTP 200, all-zero vector. Non-finite** → normalisation yields NaN → Go's encoding/json refuses to encode a NaN float → **HTTP 500 failed to encode response: json: unsupported value: NaN**. The error string is a *symptom of the JSON encoder*, which is why it reads like a serialisation bug and sent everyone looking in the wrong place.
6. Repeated timeouts degrade the Vulkan context (eventually GPU HANG / context reset). Past that point the device is effectively lost: every dispatch returns garbage instantly, so even "hi" fails in 0.06 s. Only tearing down the process (ollama stop) rebuilds the Vulkan context.

4.2 Why the numbers look “random”

Because the trigger is a property of the *input*, not of the load. A corpus with few context-filling chunks survives 144 requests; one with an oversized file wedges on request 4. The earlier observation that “5 consecutive embeds succeed on both GPU and CPU” is fully consistent — 5 *small* embeds will never trigger it, and neither will 300.

4.3 The silent corruption is the bigger problem

Same text, same model, fresh runner each time:

chars	CPU norm	GPU norm	cosine(CPU, GPU)	verdict
400	1.0000	1.0000	1.000000	match
2 000	1.0000	1.0000	1.000000	match
4 000	1.0000	1.0000	1.000000	match
8 000	1.0000	1.0000	0.984649	corrupt

And determinism, identical input repeated 4× on one fresh runner:

- **CPU:** all pairwise cosines **1.000000**. Bit-deterministic.
- **GPU:** all four vectors **all-zero** (norm 0, cosine undefined) — while returning HTTP 200.

So the GPU path at full context produces, non-deterministically, one of: a *slightly wrong* vector (cos 0.985 — passes every sanity check, quietly poisons retrieval), an *all-zero* vector (HTTP 200, poisons retrieval harder), or *NaN* (HTTP 500, at least it's loud).

Only the third is visible to a caller that checks status codes. Any index built on the Vulkan path may already contain junk vectors for its largest chunks — and those are usually the most information-dense files. Recommend a full clean reindex after applying the fix, not an incremental one.

5. Environment audit

5.1 Version — old, and the distro cannot help

```
$ ollama --version
ollama version is 0.23.4
$ rpm -q ollama
ollama-0.23.4-alt1.x86_64
$ apt-cache policy ollama
  Installed: 0.23.4-alt1:sisyphus+418061.100.1.1
  Candidate: 0.23.4-alt1:sisyphus+418061.100.1.1    <- no newer
package
```

Upstream at the time of writing is **~0.32.x stable / 0.33.0-rc2** (2026-08-22). The installed build is roughly **nine minor versions behind**, and **ALT Sisyphus has nothing newer**. Upgrading means going outside the distro (upstream install script or binary tarball) and carries its own risk — see §7 on why upgrading is *not* the recommended first move.

5.2 The Vulkan gate is broken in this build

Startup, verbatim:

```
msg="server config" env="map[... GGML_VK_VISIBLE_DEVICES: ...  
OLLAMA_VULKAN:false ...]"  
msg="inference compute" ... library=Vulkan ... description="Intel(R)  
Iris(R) Xe Graphics (TGL GT2)" type=iGPU
```

OLLAMA_VULKAN:false — and Vulkan is used anyway. The binary contains the string experimental Vulkan support disabled. To enable, set OLLAMA_VULKAN=1, so the gate exists, but in this ALT build it does not prevent Vulkan device discovery. **Setting OLLAMA_VULKAN=0 will not fix this.** That dead end is worth writing down.

5.3 Which knobs actually work

Tested by starting a throwaway second ollama serve on port 11533 with its own models directory (the system service was never restarted) and reading the inference compute line:

Env	Result	Verdict
<i>(baseline, as the service runs today)</i>	library=Vulkan	GPU used
OLLAMA_VULKAN=0	library=Vulkan (already false)	ineffective
OLLAMA_LLM_LIBRARY=cpu	library=Vulkan	ineffective
GGML_VK_VISIBLE_DEVICES=-1	library=cpu , zero Vulkan lines	works

5.4 Service configuration

```
FragmentPath=/usr/lib/systemd/system/ollama.service  
DropInPaths=(none)  
Environment=(empty)  
EnvironmentFiles=/etc/sysconfig/ollama (ignore_errors=yes)  <-  
referenced, does not exist  
OLLAMA_NUM_PARALLEL=1    OLLAMA_MAX_QUEUE=512  
OLLAMA_KEEP_ALIVE=5m
```

The unit **already** sources /etc/sysconfig/ollama optionally. Creating that file sets service-wide environment **without editing any unit file**. That is the clean seam for the durable fix.

OLLAMA_NUM_PARALLEL=1 also explains the operational pain: one embed at a time, so a wedged or 20-s-timing-out runner stalls every client behind it.

6. Upstream: known bug, still open, no fix landed

This is not local misconfiguration. The exact combination is reported upstream, repeatedly.

Closest matches — the same failure, precisely:

- [ggml-org/llama.cpp#18969](#) — “*Vulkan Backend - NaN on Intel iGPU when injecting Embeddings (f16 acc overflow)*”. Intel iGPU + Vulkan + embeddings + NaN. Root cause given as the Vulkan backend defaulting to **F16 accumulation** for `mul_mat_mat`, overflowing F16’s 65504 range. Workarounds listed: keep batch ≤ 8 (forces the `mul_mat_vec` kernel), use FP32 weights, or **switch to the CPU backend**. A runtime `GGML_VK_FORCE_F32_ACC` flag was *requested and does not exist*. Status: **open, no fix**.
- [ggml-org/llama.cpp#26044](#) — “*certain inputs make ...-Embedding return an all-NaN embedding and then permanently wedge the server; CPU output is correct*”. This is our wedge semantics verbatim: content-specific trigger, HTTP 200 with clean logs, **permanently wedged until process restart, -ngl 0 (CPU) produces the correct vector**. Attributed to “persistent GPU state”. Status: **open, no fix**.

Corroborating — Vulkan on Intel iGPU produces garbage:

- [ollama/ollama#13086](#) — Vulkan on Intel iGPU produces “absolute garbage” output, no errors logged; CPU-only works. Workaround: disable Vulkan.
- [ollama/ollama#15248](#) — Vulkan/**i915** on an Intel iGPU, corrupted/garbled output, ollama 0.20.0. Open.
- [ggml-org/llama.cpp#16684](#) — Vulkan, Intel Arc iGPU **hangs**.
- [ggml-org/llama.cpp#20201](#) — Intel iGPU + Vulkan crashes.

Same error string, other stacks (all downstream of the same encoder behaviour): [ollama#9639](#), [ollama#12921](#), [ollama#13572](#), [ollama#14657](#), [LightRAG#1870](#), [ragflow#15245](#), [dify#30890](#).

Consequence for the fix decision: the bug is in the ggml Vulkan backend and is **open with no merged fix**. “Upgrade ollama” is therefore *not* a fix — it is a gamble on an unfixed upstream defect, and it would also mean leaving the distro package. **Avoid Vulkan** is the fix.

7. Recommended fix

Single recommendation

Take the Intel iGPU out of the embedding path. The workload is a 137M-parameter embedding model; the iGPU buys nothing here. Measured: 400-char embeds are **0.41 s on GPU vs ~1.3 s on CPU**, and at full context the GPU is only “faster” (20 s vs 70 s) because it is *timing out rather than computing*. Correct-and-slower beats fast-and-silently-wrong.

Tier 1 — right now, zero restart, zero risk. Send `num_gpu: 0` on every embed:

```
{ "model": "ordis/jina-embeddings-v2-base-code",  
  "input": [...],  
  "options": { "num_gpu": 0 } }
```

Proven in this experiment: loads offloaded 0/13 layers to GPU, returns correct bit-deterministic vectors, zero i915 events across every CPU condition.

Tier 2 — client-agnostic, no restart, no unit edit. Pin CPU into the model itself so any caller gets it, including ones you cannot pass options through:

```
printf  
    'FROM ordis/jina-embeddings-v2-base-code\nPARAMETER  
    num_gpu 0\n' > Modelfile.cpu  
ollama create jina-embeddings-code-cpu -f Modelfile.cpu
```

Then point Lumen at `jina-embeddings-code-cpu`. **This tag was created during the experiment and is validated in §8 — it runs on CPU with no client-side options at all.** It shares blobs with the original (no extra disk, nothing re-pulled). Remove with `ollama rm jina-embeddings-code-cpu` if unwanted.

Tier 3 — durable, host-wide. Create `/etc/sysconfig/ollama` (the unit already sources it, so **no unit file is modified**):

```
# /etc/sysconfig/ollama
# Intel Iris Xe + Mesa Vulkan corrupts embeddings at full context
# and wedges the
# runner (i915 fence timeouts -> GPU hang). Upstream:
# llama.cpp#18969, #26044.
# NOTE: OLLAMA_VULKAN=0 does NOT work on this build - it is
# already false and
# Vulkan is used anyway. GGML_VK_VISIBLE_DEVICES=-1 is what
# actually disables it.
GGML_VK_VISIBLE_DEVICES=-1
```

Then, **at a moment when no indexing job is running**,

sudo systemctl restart ollama. Verify with: journalctl -u ollama |
grep "inference compute" → must show library=cpu. This was **not**
applied here, because restarting the service was out of scope for this
experiment.

Also do

- **Reindex clean, don't resume.** Per §4.3, existing vectors for large chunks may be all-zero or subtly wrong while having been accepted as HTTP 200.
 - **Make the client reject bad vectors.** Any embedding client should treat a zero-norm vector and a non-finite component as hard errors, not just non-200. Today zerovec sails through.
 - **Cap chunk size.** Even on CPU, keeping chunks under ~4000 characters keeps you well clear of the context-fill boundary and cuts latency by ~4x (18 s → 4.8 s at 2000 chars).
 - **Do not upgrade ollama as the fix.** §6 — upstream is unfixed, and it means leaving the distro package. Upgrade for other reasons, on its own merits, after the CPU pin is in place.
-

8. Validation of the fix

Both no-restart tiers were validated **behaviourally**, on the exact input that corrupts the GPU path on request 1. CPU and GPU have unmistakable signatures at full context — CPU takes 70–110 s and returns a unit-norm vector; GPU takes ~20 s and returns norm 0 — so latency plus vector norm identifies the backend without trusting any log line.

```
=== CPU-pinned tag, DEFAULT options (is PARAMETER num_gpu 0
honoured?) ===
tag-default #1 http=200 95.1s norm=1.0000 ->
CPU(correct)
```



```

tag-default #2          http=200  109.5s norm=1.0000 ->
CPU(correct)
tag-default #3          http=200   80.5s norm=1.0000 ->
CPU(correct)
=== plain model + options num_gpu:0 (known-good control) ===
plain+num_gpu0 #1       http=200   99.1s norm=1.0000 ->
CPU(correct)
plain+num_gpu0 #2       http=200   94.8s norm=1.0000 ->
CPU(correct)

```

- **Tier 1 (options:{"num_gpu":0}) — confirmed.** 5/5 here, and 18/18 across every CPU condition in §3. Never a fence timeout.
- **Tier 2 (PARAMETER num_gpu 0 in a Modelfile) — confirmed.** The derived tag `jina-embeddings-code-cpu` runs on CPU **with no options sent by the client at all**, 3/3. This is the one that works for callers you cannot modify.
- **Tier 3 (GGML_VK_VISIBLE_DEVICES=-1) — confirmed at the discovery layer only** (§5.3): a throwaway `ollama serve` with that variable reports `library=cpu` and logs no Vulkan device. Not end-to-end tested, because that needs a service restart.

A methodological warning, because it cost me time and will cost the next person time. My first attempt to verify Tier 2 read `load_tensors: offloaded N/13 layers to GPU` out of the journal after each request, and produced *flip-flopping, self-contradictory* answers — the same request reported 13/13 and then 0/13. The journal line lags the request and the grep window catches the *previous* load. **Do not verify GPU placement from a timed journal grep.** Use the behavioural signature above, or capture a journal cursor before the request.

Service integrity after the experiment: `NRestarts=0`,
`ActiveEnterTimestamp= Tue 2026-08-25 11:45:36` — the `ollama` service was never restarted, no unit file was touched, no model was deleted or re-pulled. The suspended `lumen index` worker was resumed and no process was left stopped.

9. What remains unproven

Stated plainly, because the fix decision should not rest on any of these:

1. **The F16-accumulation-overflow mechanism is inferred, not measured here.** I proved Vulkan+full-context → i915 fence timeout → garbage. Upstream #18969 attributes the NaN to F16 accumulation overflow. I did **not** instrument `ggml` to confirm that is

the specific arithmetic path on this hardware. It may be overflow, or it may be purely the abandoned dispatch leaving the buffer unwritten. **The fix is the same either way.**

2. **GGML_VK_DISABLE_F16=1 was not tested.** `libggml-vulkan.so` exposes it. If keeping GPU acceleration matters, that knob is worth an experiment — it might fix the corruption while retaining the iGPU. Untested, so unrecommended.
3. **The exact token threshold is bracketed, not pinned.** Fence timeouts appear between 4000 chars (clean) and 8000 chars (always fails), on this specific pseudo-code text. The real boundary is in tokens and will shift with content. I did not bisect it.
4. **CPU was not driven to the same total token volume as GPU.** CPU is ~4x slower per full-context request, so equal-volume runs were not affordable in the window. CPU is clean across **26 full-context requests** (18 in §3 + 5 in §8 + 3 in B2) and every smaller condition; GPU corrupts on **request 1** and hard-wedges by **request 8**. The contrast is decisive, but “CPU never fails, ever” is not proven — only “CPU does not fail where GPU fails immediately, repeatedly, and at a reproducible request number”.
5. **GGML_VK_VISIBLE_DEVICES=-1 was validated at device-discovery only**, not end-to-end through a real embed, because confirming it requires restarting the service (out of scope). Tiers 1 and 2 are end-to-end proven; Tier 3 is not.
6. **ollama’s GPU placement on this host looked non-deterministic.** One runner (23:13:06) loaded `GPULayers:[]` with no `gpu memory` line in the scheduler log, i.e. the Vulkan device was momentarily absent from ollama’s device list. If real, this means the *same* request can land on GPU or CPU across loads — which would make the fault appear to come and go on its own. I could not separate this from my own racy log-window artifacts (§8 warning), so it is **an observation, not a finding**.
7. **Whether the wedge is recoverable without ollama stop** was not explored (no attempt to reset the Vulkan context in place). `ollama stop` is known to work and is cheap.
8. **Concurrency was not tested as an independent variable.** This host was at `OLLAMA_NUM_PARALLEL=1` on the day of the run, so requests serialised anyway; contention changed latency 130x but no condition was run at deliberately varied parallelism. That figure is a dated observation about one machine — `bash scripts/ollama-tune.sh` reads the live value off the service and computes what this host’s CPU/RAM/model facts justify, so there is no longer any reason to carry the number in prose.
9. **Newer ollama was not tested**, because no newer package exists in ALT Sisyphus and installing upstream out-of-band was out of scope.

Whether 0.32/0.33 behaves differently on this iGPU is unknown — §6 suggests probably not.

10. **The historical journal analysis is observational.** The 12 h natural experiment (§3.4) had the `lumen index` job as an uncontrolled variable. It corroborates the controlled results; it does not stand alone.
-

Appendix: reproducing in one line

```
ollama stop ordis/jina-embeddings-v2-base-code; sleep 5
python3 - <<'EOF'
import json,urllib.request
t="def f():\n    return 1\n"*400          # ~8 kB, fills the 8192-
    token context
for i in range(8):
    b=json.dumps({"model":"ordis/jina-embeddings-v2-base-
        code","input":[t]*4}).encode()
    try:
        r=urllib.request.urlopen(urllib.request.Request(
            "http://localhost:11434/api/embed",data=b,
            headers={"Content-Type":"application/
                json"}),timeout=400)
        v=json.loads(r.read())["embeddings"][0]
        print(i, r.status, "ZERO VECTOR" if not any(v) else "ok")
    except Exception as e:
        print(i, "ERR", e)
EOF
curl -s localhost:11434/api/embed \
    -d '{"model":"ordis/jina-embeddings-v2-base-code","input":"hi"}'
# -> {"error":"failed to encode response: json: unsupported value:
    NaN"}
```

Watch `journalctl -k -f | grep -i fence` in another terminal. Add `"options":{"num_gpu":0}` to both requests and everything above passes.